



파이썬 프로그래밍

자료구조 1: 리스트(list), 튜플(tuple)

Fall Semester 2025-02

리스트 (List)

- **정의(Definition):** 리스트는 순서가 있는(Sequential) 데이터들의 모음(collection).
- **특징:**
 1. **순서 유지** → 저장한 순서대로 요소를 꺼낼 수 있음
 2. **변경 가능(Mutable)** → 요소를 추가, 삭제, 수정 가능
 3. **동적 크기 조절** → 실행 중(run-time)에도 크기를 늘리거나 줄일 수 있음
 4. **인덱싱(Indexing)** → 위치 번호(인덱스)를 이용해 요소 참조 가능
 5. 문자열처럼 **연결, 슬라이싱(slicing), 반복, 멤버 검사, 길이 구하기** 등 다양한 연산 지원
 6. **[]** 대괄호로 표현

```
[]      # 빈 리스트(empty list)
[1, 2, 3, "abc"] # 요소가 4개인 리스트
[1, 2, 3, [1, 2]] # 요소에 리스트가 포함됨
```

리스트 생성

- 빈 리스트 생성 후 요소 추가
- 빈 리스트 생성

```
변수_이름 = []  
변수_이름 = list()
```

- 리스트 요소 추가
 - ▣ append() 함수는 **리스트의 마지막에 새로운 요소들을 추가**

```
a = []  
a.append(4)           # [4]  
a.append('5')         # [4, '5']  
a.append([2, 3])      # [4, '5', [2, 3]]
```

리스트 (List)

- 리스트의 요소 개수 확인하기
 - ▣ len() 함수 사용

```
e = []      # empty list
l1 = [1, 2, 3, "abc"] # 요소가 4개인 리스트
l2 = [1, 2, 3, [1, 2]] # 요소에 리스트가 포함됨
len([])     # 0
len(e)
len(l1)
len(l2)
len([1, 2, 3, "abc"])
len([1, 2, 3, [1, 2]])
```

리스트 (List)

□ 인덱싱

- ▣ 특정(단일) 요소에 접근(access)해서 값을 읽거나 변경하는 방법
- ▣ **[인덱스]**를 사용해서 요소에 접근

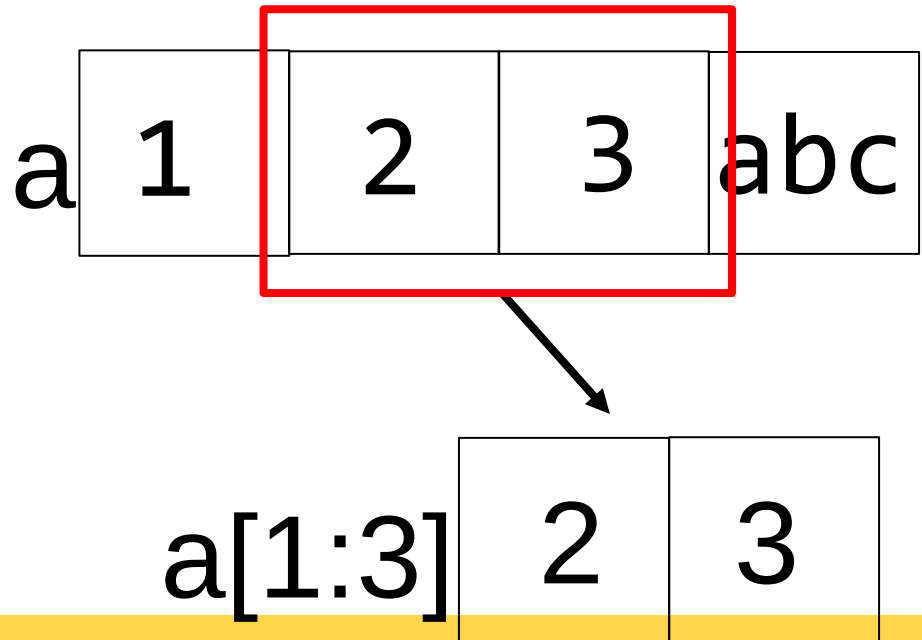
[0]	[1]	[2]	[3]
1	2	3	abc
[-4]	[-3]	[-2]	[-1]

```
l = [1, 2, 3, "abc"] # 리스트 생성
print(l[0], l[1], l[2], l[3]) # 순서대로 출력
print(l[-1], l[-2], l[-3], l[-4]) # 거꾸로 출력
```

리스트 (List)

- 슬라이싱(slicing)
 - ▣ 여러 개 요소에 접근하는 방법
 - ▣ **[시작인덱스:끝인덱스]**를 사용해서 요소들에 접근
 - ▣ 시작인덱스에 해당되는 요소부터 끝인덱스 - 1 까지의 요소들을 포함하는 리스트 참조

```
a = [1, 2, 3, "abc"]  
a[1:3]
```



리스트 (List)

□ 슬라이싱(slicing)

```
lst1 = [1, 2, 3, "abc", 4, 5]
lst1[2:4]      # [3, "abc"]
lst1[:3]       # [1, 2, 3]
lst1[2:]       # [3, "abc", 4, 5]
lst1[:]        # [1, 2, 3, "abc", 4, 5]
```

□ 확장(extended) 슬라이싱

- [시작인덱스 : 끝인덱스 : 스텝]
- 시작인덱스부터 끝인덱스 - 1까지

```
lst1 = [1, 2, 3, "abc", 4, 5]
lst1[2:5:2]    # [3, 4]
lst1[2:6:2]    # [3, 4]
lst1[:3:2]     # [1, 3]
lst1[2::2]     # [3, 4]
```

리스트 (List)

□ 연결하기

- ▣ + 연산자를 이용해서 두 개 이상의 리스트를 연결해서 새로운 리스트를 생성할 수 있음

```
a = [1, 2, 3]
b = ["abc", 4, 5]
c = a + b # 두 개의 리스트를 붙여서 c에 저장
print(c)  # [1, 2, 3, "abc", 4, 5]
```

□ 반복하기

- ▣ * 연산자를 이용해서 요소들이 반복되는 리스트 생성

```
a = [1, 2]
b = a * 4
print(b)          # [1, 2, 1, 2, 1, 2, 1, 2]
```

리스트 (List)

□ 리스트 요소 수정

▣ 특정 요소의 값을 변경

■ 리스트이름[인덱스] = 새로운 값

```
a = [1, 2, 3, "abc"]  
a[2] = 4          # [1, 2, 4, "abc"]  
a[3] = ["ab", 'a', "def"] # [1, 2, 4, ["ab", 'a', "def"]]
```

▣ 여러 개 요소를 새로운 값으로 치환하고자 할 때에는 아래와 같이 슬라이싱 방법을 사용해서 수정

▣ 리스트이름[시작인덱스:끝인덱스] = 새로운 값

```
a = [1, 2, 3, "abc"]  
a[1:2] = ['g', 'h'] # [1, 'g', 'h', 3, "abc"]  
a[2:4] = ["ab", 'a', "def"]  
          # [1, 'g', "ab", 'a', "def", "abc"]
```

리스트 (List)

- 리스트 요소 삭제
 - ▣ del 명령을 이용해서 특정 요소 삭제
 - ▣ del 삭제하고 싶은 요소 또는 del(삭제하고 싶은 요소)

```
a = [1, 2, 3, "abc", 'g', 'h']  
del a[1] # [1, 3, "abc", 'g', 'h']
```

- ▣ 범위 삭제
- ▣ 리스트이름[시작인덱스:끝인덱스] = []

```
a = [1, 2, 3, "abc", 'g', 'h']  
a[1:2] = [] # [1, 3, "abc", 'g', 'h']  
a[2:4] = [] # [1, 3, 'h']  
del(a[0:2]) # ['h']
```

리스트 (List)

□ 리스트 요소 제거

- ▣ 리스트의 요소를 제거하려면 remove(x) 활용
- ▣ 리스트에서 첫 번째로 나오는 x 제거

```
a = [1, 3, 5, 7, 9, 7, 5, 3, 1]
a.remove(5)    # [1, 3, 7, 9, 7, 5, 3, 1]
a.remove(3)    # [1, 7, 9, 7, 5, 3, 1]
a.remove(3)    # [1, 7, 9, 7, 5, 1]
```

□ 리스트에 포함된 요소 x의 개수 세기

- ▣ count(x) 함수 활용
- ▣ 리스트에서 x의
개수 반환

```
a = [1, 5, 7, 5, 7, 5, 3, 1]
a.count(5)     # 3
a.count(3)     # 1
a.count(7)     # 2
```

리스트 (List)

- 요소의 존재 여부 확인하기

- ▣ in 사용

```
a = [4, 8, 7, 2, 1]
8 in a           # True
9 in a           # False
```

- 인덱스 확인하기

```
a = [4, 8, 7, 2, 1]
a.index(8)        # 1
a.index(1)        # 4
```

리스트 (List)

- 리스트에 요소 삽입
 - ▣ 요소 삽입은 `insert(x, y)` 함수 활용. `x` index 위치에 `y` 삽입

```
a = [4, 8, 7, 2, 1]
a.insert(2, 5) # [4, 8, 5, 7, 2, 1]
a.insert(1, [1, 6])
# [4, [1, 6], 8, 5, 7, 2, 1]
```

리스트 (List)

□ 리스트 요소 정렬(sort)

▣ sort()함수 활용

```
a = [4, 8, 7, 2, 1]
a.sort()           # [1, 2, 4, 7, 8]
a.append('a')      # [1, 2, 4, 7, 8, 'a']
a.sort()           # Error 발생: 문자열과 정수를 비교함

b = ['g', 'o', 'o', 'd']
b.sort()           # ['d', 'g', 'o', 'o']
```

□ 리스트 요소 뒤집기

▣ reverse()함수 활용

```
a = [4, 8, 7, 2, 1]
a.reverse()         # [1, 2, 7, 8, 4]
```

문자열을 잘라서 리스트 생성하기: split()

□ split()은 문자열을 **구분자(separator)** 기준으로 잘라 리스트로 만들어 주는 함수이다.

□ split() 함수의 사용법에는 여러 가지가 있다.

1. 문자열.split(구분문자)

■ 지정한 **구분문자(separator)** 를 기준으로 문자열을 나누어 리스트로 반환한다.

■ `text = "apple, banana, cherry"`

■ `result = text.split(",")` → # 출력: ['apple', 'banana', 'cherry']

2. 문자열.split(구분문자, 횟수)

■ 지정한 **구분문자**로 **최대 횟수만큼** 문자열을 나눈다.
(나누는 횟수를 제한할 수 있음)

■ `text = "apple, banana, cherry"`

■ `result = text.split(", ", 1)` → # 출력: ['apple', 'banana cherry']

문자열을 잘라서 리스트 생성하기: split()

- split()은 문자열을 **구분자(separator)** 기준으로 잘라 리스트로 만들어 주는 함수이다.
- split() 함수의 사용법에는 여러 가지가 있다.

3. 문자열.split()

- 공백 문자(스페이스, 탭, 줄바꿈 등) 를 기준으로 나눈다.
연속된 공백은 하나로 취급한다.
 - `text = "apple, banana, cherry"`
 - `result = text.split()` → # 출력: ['apple,', 'banana,', 'cherry']

4. 문자열.split(maxsplit=횟수)

- 기본 구분자(공백)를 기준으로 하지만, maxsplit 인자를 사용하여 나누는 횟수를 제한할 수 있다.
 - `text = "apple banana cherry grape"`
 - `result = text.split(maxsplit=2)` → # 출력: ['apple', 'banana', 'cherry grape']

for 반복문과 리스트 사용하기

- 리스트는 순서가 있는 객체
- for 반복문을 이용해서 리스트의 각 요소에 접근 가능

```
1  a = [4, 8, 7, 2, 1]
2  for n in a:
3      print(n)
4
5  l = [1, 'g', 'h', 3, "abc"]
6  for n in l:
7      print(n)
8
9  # 리스트에서 4보다 큰 정수만 출력
10 for n in a:
11     if n > 4:
12         print(n)
```

튜플 (Tuple)

- 튜플은 리스트와 비슷하게 순서가 있는 요소들의(Seq. Items) 목록
- 특징:
 1. **순서 유지** → 저장된 순서대로 요소를 꺼낼 수 있음
 2. **변경 불가능(Immutable)** → 한 번 생성하면 요소를 수정, 추가, 삭제할 수 없음
(리스트와의 가장 큰 차이)
 3. 리스트와 마찬가지로 **연결, 슬라이싱, 반복, 멤버 검사, 길이 구하기** 등의 연산 가능
단, 연산 결과는 **새로운 튜플**로 생성됨
 4. <소괄호 ()>로 표현 (괄호 생략(omitted) 가능)
 5. 요소가 하나인 튜플은 ,(кома) 필수
예: (10,)는 튜플, (10)은 그냥 정수

튜플 (Tuple)

□ 예시

```
() # 빈 튜플(empty tuple)

(1, 2, 3, "abc") # 요소가 4개인 튜플

(1, 2, 3, (1, 2)) # 요소에 튜플 포함 가능

(1, 2, 3, [1, 2]) # 요소에 리스트 포함 가능

tuple1 = (2, 3)

tuple2 = 2, 3 # 괄호 생략(omitted) 가능

tuple3 = 2, 3, [1, 2], (1, 2)
```

튜플 (Tuple)

- 요소가 한 개인 튜플을 생성할 때에는 반드시 콤마(',')로 끝내야 함

```
tuple1 = (2,)      # 튜플 생성
tuple2 = 2,        # 튜플 생성
tuple3 = ("hello",) # 튜플
tuple4 = "hello",  # 튜플
tuple5 = (2)        # 정수
tuple6 = ("hello")  # 문자열
```

튜플 (Tuple)

□ 튜플의 요소 개수 확인하기

▣ `len()` 함수 사용

```
e = ()
t1 = (1, 2, 3, "abc")
t2 = (1, 2, 3, (1, 2))
t3 = 1, 2, 3, [1, 2], (1, 2)
len(())      # 0
len(e)
len(t1)
len(t2)
len((1, 2, 3, "abc"))
len((1, 2, 3, [1, 2]))
```

튜플 (Tuple)

□ 인덱싱

- ▣ 특정(단일) 요소에 접근(access)해서 값을 읽거나 변경하는 방법
- ▣ [인덱스]를 사용해서 요소에 접근

[0]	[1]	[2]	[3]
1	2	3	abc
[-4]	[-3]	[-2]	[-1]

```
a = (1, 2, 3, "abc") # 튜플 생성
print(a[0], a[1], a[2], a[3]) # 순서대로 출력
print(a[-1], a[-2], a[-3], a[-4]) # 거꾸로 출력
```

튜플 (Tuple)

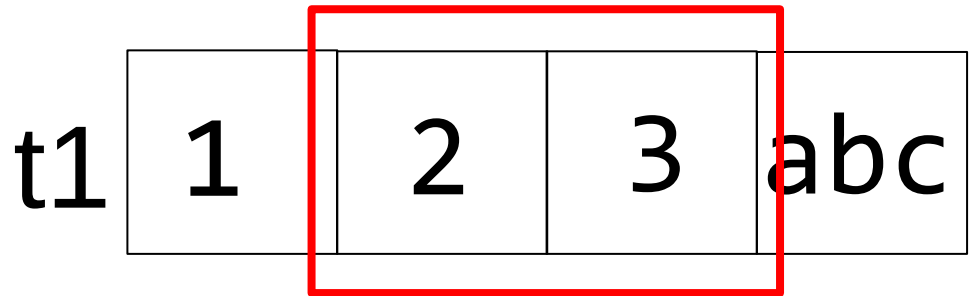
□ 슬라이싱(slicing)

▣ 여러 개 요소에 접근하는 방법

▣ [시작인덱스 : 끝인덱스]를 사용해서 요소들에 접근

▣ 시작인덱스에 해당되는 요소부터 끝인덱스 - 1 까지의 요소들을 포함하는 튜플 생성

```
t1 = (1, 2, 3, "abc")  
t2 = t1[1:3]
```



t2 = t1[1:3]

2	3
---	---

튜플 (Tuple)

□ 슬라이싱(slicing)

```
a = (1, 2, 3, "abc", 4, 5)
a[2:4]      # (3, "abc")
a[:3]       # (1, 2, 3)
a[2:]       # (3, "abc", 4, 5)
a[:]        # (1, 2, 3, "abc", 4, 5)
```

□ 확장(extended) 슬라이싱

■ [시작인덱스:끝인덱스:스텝]

■ 시작인덱스부터 끝인덱스 - 1까지 스텝만큼 건너뛰면서 튜플 참조

```
a = (1, 2, 3, "abc", 4, 5)
a[2:5:2]     # (3, 4)
a[2:6:2]     # (3, 4)
a[:3:2]      # (1, 3)
a[2::2]      # (3, 4)
```



튜플 (Tuple)

□ 연결하기

- ▣ + 연산자를 이용해서 두 개 이상의 튜플을 연결해서 새로운 튜플을 생성할 수 있음

```
t1 = (1, 2, 3)
t2 = ("abc", 4, 5)
t = t1 + t2 # 두 개 튜플을 붙여서 t에 저장
print(t)    # (1, 2, 3, "abc", 4, 5)
```

□ 반복하기

- ▣ * 연산자를 이용해서 요소들이 반복되는 튜플 생성

```
a = (1, 2)
b = a * 4
print(b)          # (1, 2, 1, 2, 1, 2, 1, 2)
```

실습문제 1

□ 문제

- ▣ 다음과 같은 문자열을 가진 두 개의 리스트 list1, list2가 있을 경우 list1과 list2의 조합을 이용하여 다음과 같이 출력하시오.

```
list1 = ['I like', 'I love']  
list2 = ['pancakes.', 'kiwi juice.', 'espresso.']
```

□ 실행 결과 예시

```
I like pancakes.  
I like kiwi juice.  
I like espresso.  
I love pancakes.  
I love kiwi juice.  
I love espresso.
```

실습문제 2

□ 문제

- 사용자로부터 5개의 수를 입력 받은 후 다음과 같이 입력된 값들의 합, 평균, 최댓값, 최솟값을 출력하는 프로그램을 작성하시오.

□ 실행 결과 예시

```
Enter 5 numbers: 12,-4,10,2,1
Sum : 21
Average : 4.2
Maximum value : 12
Minimum value : -4
```

실습문제 3

□ 문제

사용자로부터 n 개의 값을 입력받은 후 입력된 값들의 평균(mean)과 표준편차(standard deviation: σ 로 표기함)를 구하시오. 입력된 값들을 x_i 라고 할 때, 표준 편차(σ)를 구하는 식은 다음과 같다.

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - mean)^2}{n}}$$

□ 실행 결과 예시

```
Enter integers separated by spaces: 2 4 5 5 7
Total sum: 23
Average (mean): 4.60
Standard deviation: 1.62
```

실습문제 4

□ 문제

- 문자열 src가 'aaaabbbb'와 같을 경우, 이를 압축하는 방법으로 'a4b3'과 같이 문자와 그 반복 횟수를 표시하는 방법이 있을 것이다. 문자열 'aaaabccccaaaaacccfg'에서 같은 문자가 있을 경우 이를 'a4b1c4a5c3f1g1' 같은 output 문자열로 압축해 보자.

□ 실행 결과 예시

```
src = ''  
output = ''
```

```
src = 'a'  
output = 'a1'
```

```
src = 'aaaabbbb'  
output = 'a4b3'
```

```
src = 'aaaabccccaaaaacccfg'  
output = 'a4b1c4a5c3f1g1'
```

실습문제 5

□ 문제

- 주어진 튜플 tup에서 한 번 이상 나타나는 원소를 찾아 집합 dup에 넣어서 이를 출력하라. 다음의 튜플에서 1, 2, 4는 2번 9는 3번 나타난다. 따라서 결과는 {1, 2, 4, 9}가 된다.

□ 실행 결과 예시

```
Original tuple: (1, 2, 5, 4, 3, 2, 9, 1, 4, 7, 8, 9, 9)
Duplicated elements: [1, 2, 4, 9]
```

실습문제 6

□ 문제

- 어떤 튜플이 주어졌을 때, 처음에는 튜플 전체를 출력하여라. 그 다음에는 마지막 한 요소를 제외하고 출력하며, 그 다음 단계에는 마지막 두 요소를 제외하고 출력하는 코드를 슬라이싱을 이용하여 작성해 보라.

□ 실행 결과 예시

```
Enter the elements of the tuple separated by spaces: -1 2 7 5 9 8  
  
(-1, 2, 5, 7, 8, 9)  
(-1, 2, 5, 7, 8)  
(-1, 2, 5, 7)  
(-1, 2, 5)  
(-1, 2)  
(-1,)
```

실습문제 7

□ 문제

- ▣ 정수 한 개(n)를 함수의 인자로 받고, 그 정수의 모든 약수를 리스트로 구성해서 반환하는 함수(createDivisorsList)를 구현. 이 함수를 이용해서 사용자로 부터 1~1000 정수 중에서 하나를 입력 받고, 그것의 모든 약수들의 합을 계산해서 출력하는 프로그램 작성
- ▣ 약수
 - 정수 n 의 약수를 찾는 방법은 1~ n 까지의 정수를 n 으로 나눠서 나머지가 0인지 확인

□ 요구사항

- ▣ 사용자로 부터 입력 받는 값은 1~1000 정수로 가정
- ▣ createDivisorsList 함수를 호출하는 코드에서 리스트의 내용을 화면에 출력함

□ 실행 결과 예시

```
Enter an integer less than or equal to 1000: 750
[1, 2, 3, 5, 6, 10, 15, 25, 30, 50, 75, 125, 150, 250, 375, 750]
Total sum of divisors: 1872
```

실습문제 8

□ 문제

- 도형 정보를 담고 있는 튜플의 요소들을 이용해서 도형의 면적을 계산해서 출력하는 프로그램 작성
- 도형 정보를 담고 있는 튜플의 예시
 - ("사각형", 30, 20, "원", 10)

□ 요구사항

- 튜플에 있는 도형의 개수는 정해져 있지 않음
- 원주율은 math.pi 사용 (import math 필요)
- calcAndPrintArea() 함수는 튜플을 인자로 받고 면적을 계산해서 화면에 출력
- 튜플 예시: ("rectangle", 30, 20, "circle", 10, "rectangle", 20, 40, "rectangle", 10, 10, "circle", 20)사용

□ 실행 결과 예시

```
rectangle, 30, 20, 600
circle, 10, 314.1592653589793
rectangle, 20, 40, 800
rectangle, 10, 10, 100
circle, 20, 1256.6370614359173
```

